

DSA Sheet 4 - Problems 61 to 79

--- Problem 61: Cousins in Binary Tree ---

Problem Statement

Given a binary tree having distinct node values, and two node values a and b, check whether the two nodes are cousins or not.

Two nodes are said to be cousins if:

They are at the same level (same depth).

They have different parents.

Return true if the nodes are cousins, otherwise return false.

Input Format

The first line contains the root of the binary tree.

The next line contains two integers a and b representing the node values to be checked.

Output Format

Print true if the given nodes are cousins.

Otherwise, print false.

Constraints

1 <= number of nodes <= 10^5

1 <= node->data <= 10^5

1 <= a, b <= 10^5

Example 1

Input

root = [1, 2, 3]

a = 2, b = 3

(Tree: root 1, left child 2, right child 3)

Output

false

Explanation

Nodes 2 and 3 are at the same level, but they have the same parent (1). Hence, they are not cousins.

Example 2

Input

root = [1, 2, 3, 4, null, null, 5]

a = 4, b = 5

(Tree: 1 with children 2 and 3; 4 is left child of 2; 5 is right child of 3)

Output

true

Explanation

Nodes 4 and 5 are at the same depth (level 2) and have different parents (2 and 3). Hence, they are cousins.

--- Problem 62: Spiral Order Traversal of Binary Tree ---

Problem Statement

Given a binary tree, the task is to find the spiral order traversal of the tree and return the list containing the elements.

In Spiral Order Traversal:

Start from level 0 (root level) and print nodes from right to left.

For the next level, print nodes from left to right.

Continue alternating the direction at every level.

Input Format

The input contains the root of the binary tree represented as a list.

Output Format

Return a list containing the spiral order traversal of the binary tree.

Constraints

1 <= number of nodes <= 10^5

0 <= node->data <= 10^5

DSA Sheet 4 - Problems 61 to 79

Example 1

Input

root = [1, 3, 2]

Output

[1, 3, 2]

Explanation

Start with root node 1 (level 0, right to left).

Move to level 1 and print nodes from left to right.

Final spiral order traversal: [1, 3, 2]

Example 2

Input

root = [10, 20, 30, 40, 60]

Output

[10, 20, 30, 60, 40]

Explanation

Print root 10 (level 0, right to left).

Print level 1 from left to right: 20, 30

Print level 2 from right to left: 60, 40

Final result: [10, 20, 30, 60, 40]

--- Problem 63: Even vs Odd Level Sum Difference ---

Problem Statement

Given a Binary Tree, find the difference between the sum of node values at even levels and the sum of node values at odd levels.

The root node is considered to be at level 1 (odd level).

Difference = (Sum of even level nodes) - (Sum of odd level nodes)

Input Format

The input represents the nodes of a binary tree.

Output Format

Print a single integer representing the difference between the sum of even level nodes and odd level nodes.

Constraints

1 <= number of nodes <= 10⁵

1 <= node data <= 10⁵

Example 1

Input

```
  1
 / \
2   3
```

Output

4

Explanation

Odd level nodes: 1

Even level nodes: 2 + 3 = 5

Difference = 5 - 1 = 4

Example 2

Input

```
  10
 / \
20  30
 / \
40  60
```

Output

DSA Sheet 4 - Problems 61 to 79

-60

Explanation

Odd level nodes: $10 + 40 + 60 = 110$

Even level nodes: $20 + 30 = 50$

Difference = (even) - (odd) = $50 - 110 = -60$

--- Problem 64: Coin Change ---

Problem Statement

You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Constraints

$1 \leq \text{coins.length} \leq 12$

$1 \leq \text{coins}[i] \leq 2^{31} - 1$

$0 \leq \text{amount} \leq 10^4$

Input Format

First line contains the integer array coins.

Second line contains the integer amount.

Output Format

Print a single integer representing the minimum number of coins required to make the amount.

If it is not possible, print -1.

Example 1

Input

coins = [1, 2, 5]

amount = 11

Output

3

Explanation

11 can be formed using: $5 + 5 + 1$

So, the minimum number of coins required is 3.

Example 2

Input

coins = [2]

amount = 3

Output

-1

Explanation

It is not possible to form amount 3 using coin 2 only.

Example 3

Input

coins = [1]

amount = 0

Output

0

Explanation

No coins are needed to make amount 0.

--- Problem 65: 0/1 Knapsack ---

Problem Statement

Given two arrays, val[] and wt[], where each element represents the value and weight of an item respectively, and an integer W representing the maximum capacity of the knapsack, find the maximum total value that can be included in the knapsack such that the

DSA Sheet 4 - Problems 61 to 79

total value obtained is maximum without exceeding the capacity W .

You can either include an item completely or exclude it entirely - fractional selection of items is not allowed.

Each item is available only once.

Constraints

$1 \leq \text{val.size()} = \text{wt.size()} \leq 10^3$

$1 \leq W \leq 10^3$

$1 \leq \text{val}[i] \leq 10^3$

$1 \leq \text{wt}[i] \leq 10^3$

Input Format

First line contains an integer W representing the capacity of the knapsack.

Second line contains the array $\text{val}[]$ representing values of items.

Third line contains the array $\text{wt}[]$ representing weights of items.

Output Format

Print a single integer representing the maximum value that can be obtained.

Example 1

Input

$W = 4$

$\text{val}[] = [1, 2, 3]$

$\text{wt}[] = [4, 5, 1]$

Output

3

Explanation

Only the third item can be included because its weight is 1, which is within the capacity $W = 4$.

Maximum obtainable value = 3.

Example 2

Input

$W = 5$

$\text{val}[] = [10, 40, 30, 50]$

$\text{wt}[] = [5, 4, 6, 3]$

Output

50

Explanation

The item with value 50 and weight 3 gives the maximum value within the knapsack capacity.

Hence, the answer is 50.

--- Problem 66: House Robber (Uniform Amount) ---

Problem Statement

Given a street of N houses (a row of houses), each house contains K amount of money.

A thief wants to steal money from these houses, but he cannot rob two adjacent houses.

Find the maximum amount of money the thief can rob.

Constraints

$1 \leq N, K \leq 10^3$

Input Format

First line contains an integer N representing the number of houses.

Second line contains an integer K representing the amount of money in each house.

Output Format

Print a single integer representing the maximum money that can be robbed.

Example 1

Input

DSA Sheet 4 - Problems 61 to 79

N = 5

K = 10

Output

30

Explanation

The thief can rob: 1st house = 10, 3rd house = 10, 5th house = 10

Total money robbed = $10 + 10 + 10 = 30$

So, the maximum amount that can be robbed is 30.

Example 2

Input

N = 2

K = 12

Output

12

Explanation

Since the thief cannot rob two adjacent houses, he can rob only one house.

Maximum money robbed = 12.

--- Problem 67: Egg Dropping Puzzle ---

Problem Statement

You are given n identical eggs and access to a building with k floors numbered from 1 to k .

There exists a floor f such that:

Any egg dropped from a floor higher than f will break.

Any egg dropped from floor f or lower will not break.

Rules:

A broken egg cannot be used again.

An unbroken egg can be reused.

The effect of a fall is the same for all eggs.

If an egg does not break from a certain floor, it will not break from any floor below it.

If an egg breaks from a certain floor, it will break from any floor above it.

Find the minimum number of moves required to determine the value of f with certainty in the worst case.

Constraints

$1 \leq n \leq 1000$

$1 \leq k \leq 1000$

Input Format

First line contains an integer n representing the number of eggs.

Second line contains an integer k representing the number of floors.

Output Format

Print a single integer representing the minimum number of moves required in the worst case.

Example 1

Input

$n = 2$

$k = 36$

Output

8

Explanation

With 2 eggs and 36 floors, 8 moves are sufficient to determine the highest safe floor in the worst case.

Example 2

Input

$n = 2$

DSA Sheet 4 - Problems 61 to 79

k = 28

Output

7

Explanation

Using 2 eggs and 28 floors, the minimum number of moves required in the worst case is 7.

--- Problem 68: Maximum Sum Non-Adjacent Elements ---

Problem Statement

Given an array `arr[]` containing positive integers, find the maximum possible sum of a subsequence such that no two elements chosen in the subsequence are adjacent in the array.

Constraints

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^5$

Input Format

First line contains the array `arr[]`.

Output Format

Print a single integer representing the maximum sum possible without selecting adjacent elements.

Example 1

Input

`arr[] = [5, 5, 10, 100, 10, 5]`

Output

110

Explanation

Select elements at indices 0, 3, and 5: $5 + 100 + 5 = 110$

This is the maximum sum possible without taking adjacent elements.

Example 2

Input

`arr[] = [3, 2, 7, 10]`

Output

13

Explanation

Select elements 3 and 10: total sum = 13.

--- Problem 69: Longest Increasing Subsequence (Length) ---

Problem Statement

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

A subsequence is a sequence that can be derived from an array by deleting some or no elements without changing the order of the remaining elements.

Constraints

$1 \leq \text{nums.length} \leq 2500$

$-10^4 \leq \text{nums}[i] \leq 10^4$

Input Format

First line contains the integer array `nums`.

Output Format

Print a single integer representing the length of the longest strictly increasing subsequence.

Example 1

Input

`nums = [10, 9, 2, 5, 3, 7, 101, 18]`

Output

4

DSA Sheet 4 - Problems 61 to 79

Explanation

One longest increasing subsequence is: [2, 3, 7, 101]

Length of the subsequence = 4.

Example 2

Input

nums = [7, 7, 7, 7, 7, 7, 7]

Output

1

Explanation

All elements are equal, so the longest strictly increasing subsequence contains only one element.

--- Problem 70: Largest Divisible Subset ---

Problem Statement

Given a set of distinct positive integers nums, return the largest divisible subset such that for every pair (answer[i], answer[j]) in the subset:

answer[i] % answer[j] == 0, or answer[j] % answer[i] == 0

If there are multiple possible solutions, return any one of them.

Constraints

1 <= nums.length <= 1000

1 <= nums[i] <= 2 * 10⁹

All integers in nums are unique.

Input Format

First line contains the integer array nums.

Output Format

Print the largest divisible subset.

Example 1

Input

nums = [1, 2, 3]

Output

[1, 2]

Explanation

[1, 2] is a valid subset because 2 % 1 == 0.

[1, 3] is also valid because 3 % 1 == 0.

Hence, any one valid largest divisible subset can be returned.

Example 2

Input

nums = [1, 2, 4, 8]

Output

[1, 2, 4, 8]

Explanation

Every pair in the subset satisfies the divisibility condition.

--- Problem 71: Matrix Chain Multiplication ---

Problem Statement

Given an array arr[] which represents the dimensions of a sequence of matrices, where the i-th matrix has dimensions (arr[i-1] x arr[i]) for i >= 1.

Find the most efficient way to multiply these matrices together. The efficient way is the one that involves the minimum number of scalar multiplications.

Constraints

2 <= arr.size() <= 100

1 <= arr[i] <= 200

DSA Sheet 4 - Problems 61 to 79

Input Format

First line contains the integer array `arr[]`.

Output Format

Print a single integer representing the minimum number of scalar multiplications required.

Example 1

Input

`arr[] = [2, 1, 3, 4]`

Output

20

Explanation

There are 3 matrices: M1 is 2x1, M2 is 1x3, M3 is 3x4.

Parenthesization (M1 x M2) x M3 costs $(2*1*3) + (2*3*4) = 6 + 24 = 30$.

Parenthesization M1 x (M2 x M3) costs $(1*3*4) + (2*1*4) = 12 + 8 = 20$.

Minimum cost = 20.

Example 2

Input

`arr[] = [1, 2, 3, 4, 3]`

Output

30

Explanation

Optimal parenthesization gives the minimum multiplication cost of 30.

--- Problem 72: Edit Distance ---

Problem Statement

Given two strings `word1` and `word2`, return the minimum number of operations required to convert `word1` into `word2`.

You have the following three operations permitted on a word:

Insert a character

Delete a character

Replace a character

Constraints

$0 \leq \text{word1.length}, \text{word2.length} \leq 500$

`word1` and `word2` consist of lowercase English letters.

Input Format

First line contains string `word1`.

Second line contains string `word2`.

Output Format

Print a single integer representing the minimum number of operations required.

Example 1

Input

`word1 = "horse"`

`word2 = "ros"`

Output

3

Explanation

One possible conversion: `horse` -> `rorse` (replace 'h' with 'r'); `rorse` -> `rose` (remove 'r'); `rose` -> `ros` (remove 'e').

Minimum operations required = 3.

Example 2

Input

`word1 = "intention"`

`word2 = "execution"`

DSA Sheet 4 - Problems 61 to 79

Output

5

Explanation

Minimum number of operations required is 5.

--- Problem 73: Wildcard Pattern Matching ---

Problem Statement

Given two strings `pat` and `txt` which may be of different sizes, return `true` if the wildcard pattern `pat` matches with `txt`, otherwise return `false`.

The wildcard pattern can include the characters:

? matches any single character.

* matches any sequence of characters (including the empty sequence).

Note: The matching should cover the entire text (`txt`), not just a part of it.

Constraints

$1 \leq \text{pat.length}, \text{txt.length} \leq 10^3$

Input Format

First line contains the string `txt`.

Second line contains the string `pat`.

Output Format

Print `true` if the pattern matches the text, otherwise print `false`.

Example 1

Input

`txt = "abcde"`

`pat = "a?c*"`

Output

`true`

Explanation

'a' matches 'a', '?' matches 'b', 'c' matches 'c', '*' matches "de".

Example 2

Input

`txt = "baaabab"`

`pat = "a*ab"`

Output

`false`

Explanation

The pattern does not completely match the text string, so the answer is `false`.

--- Problem 74: Stock Buy and Sell II (Multiple Transactions) ---

Problem Statement

Given an array `arr[]` representing the stock price on each day, the task is to find the maximum total profit if we can buy and sell the stocks any number of times.

Note:

We can only sell a stock after buying it.

We cannot hold multiple stocks at the same time.

Constraints

$2 \leq \text{arr.size()} \leq 10^6$

$0 \leq \text{arr}[i] \leq 10^6$

Input Format

First line contains the integer array `arr[]`.

Output Format

Print a single integer representing the maximum profit.

DSA Sheet 4 - Problems 61 to 79

Example 1

Input

arr[] = [100, 180, 260, 310, 40, 535, 695]

Output

865

Explanation

Buy on day 0 and sell on day 3: profit = 310 - 100 = 210

Buy on day 4 and sell on day 6: profit = 695 - 40 = 655

Total profit = 210 + 655 = 865

Example 2

Input

arr[] = [4, 2, 2, 4]

Output

2

Explanation

Buy at price 2, sell at price 4: profit = 2

--- Problem 75: Best Time to Buy and Sell Stock with Transaction Fee ---

Problem Statement

You are given an array prices[] where prices[i] is the price of a given stock on the i-th day, and an integer fee representing a transaction fee.

Find the maximum profit you can achieve.

You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

Note:

You may not engage in multiple transactions simultaneously.

You must sell the stock before buying again.

The transaction fee is charged only once for each buy-and-sell transaction.

Constraints

$1 \leq \text{prices.length} \leq 5 * 10^4$

$1 \leq \text{prices}[i] \leq 5 * 10^4$

$0 \leq \text{fee} < 5 * 10^4$

Input Format

First line contains the integer array prices[].

Second line contains the integer fee.

Output Format

Print a single integer representing the maximum profit.

Example 1

Input

prices = [1, 3, 2, 8, 4, 9]

fee = 2

Output

8

Explanation

Buy at 1, sell at 8: profit = (8 - 1) - 2 = 5

Buy at 4, sell at 9: profit = (9 - 4) - 2 = 3

Total profit = 8

Example 2

Input

prices = [1, 3, 7, 5, 10, 3]

fee = 3

DSA Sheet 4 - Problems 61 to 79

Output

6

Explanation

Buy at 1, sell at 10: profit = (10 - 1) - 3 = 6

--- Problem 76: Number of Longest Increasing Subsequences ---

Problem Statement

Given an integer array arr, return the number of longest increasing subsequences.

Note that the sequence has to be strictly increasing.

Constraints

$1 \leq n \leq 2000$

$1 \leq \text{arr}[i] \leq 10^5$

Input Format

First line contains the integer array arr.

Output Format

Print a single integer representing the number of longest increasing subsequences.

Example 1

Input

arr = [1, 3, 5, 4, 7]

Output

2

Explanation

The length of the longest increasing subsequence is 4.

The two longest increasing subsequences are: [1, 3, 4, 7] and [1, 3, 5, 7]

So, the answer is 2.

Example 2

Input

arr = [2, 2, 2, 2, 2]

Output

5

Explanation

Each individual element forms a strictly increasing subsequence of length 1.

There are 5 such subsequences of maximum length.

--- Problem 77: Print n to 1 Using Recursion ---

Problem Statement

Given an integer n, print numbers from n to 1 using recursion.

Constraints

$1 \leq n \leq 10^5$

Input Format

First line contains an integer n.

Output Format

Print the numbers from n to 1 separated by spaces.

Example 1

Input

n = 8

Output

8 7 6 5 4 3 2 1

Explanation

Print n, then recursively print from n-1 down to 1.

DSA Sheet 4 - Problems 61 to 79

Example 2

Input

n = 1

Output

1

Explanation

Since n is already 1, print 1.

--- Problem 78: Sum of First n Natural Numbers (Recursion) ---

Problem Statement

Given a number n, find the sum of the first n natural numbers using recursion.

Constraints

$1 \leq n \leq 10^5$

Input Format

First line contains an integer n.

Output Format

Print a single integer representing the sum of the first n natural numbers.

Example 1

Input

n = 5

Output

15

Explanation

$5 + 4 + 3 + 2 + 1 = 15$

Example 2

Input

n = 1

Output

1

Explanation

The sum of the first natural number is 1.

--- Problem 79: Sort a Stack ---

Problem Statement

Given a stack of integers st[], sort the stack in ascending order such that:

The smallest element is at the bottom.

The largest element is at the top.

Constraints

$1 \leq \text{st.size()} \leq 10^3$

$0 \leq \text{stack elements} \leq 10^3$

Input Format

First line contains the stack elements st[].

Output Format

Print the sorted stack where the top element appears first.

Example 1

Input

st[] = [1, 2, 3]

Output

[3, 2, 1]

DSA Sheet 4 - Problems 61 to 79

Explanation

Smallest element 1 is at the bottom; largest element 3 is at the top.

Printing from top to bottom: [3, 2, 1]

Example 2

Input

st[] = [41, 3, 32, 2, 11]

Output

[41, 32, 11, 3, 2]

Explanation

After sorting ascending (bottom to top): bottom = 2, top = 41.

Stack from top to bottom: [41, 32, 11, 3, 2]